

Q2 | Compare abstract classes and interfaces in terms of multiple inheritance. When would you prefer to use an abstract class and when an interface?

Ans: Abstract class: Java doesn't support multiple inheritance with abstract classes. A class can only extend one abstract class.

Code

```
abstract class Device {}  
abstract class Machine {}  
  
// class smartphone extends Device, Machine {} // Not possible  
class smartphone extends Device {} // one class.
```

Interfaces: Java supports multiple inheritance through interfaces. A class can implement multiple interfaces simultaneously. Since interfaces traditionally don't hold state or implementation, there is no ambiguity, and the child class provides the logic.

Code:

```
interface Camera { void clicked(); }  
interface MusicPlayer { void playing(); }  
  
class smartphone implements Camera, MusicPlayer {  
    public void clicked { System.out.println("click"); }  
    public void playing { System.out.println("playing"); }  
}
```

Difference between Abstract Class and Interface

Feature	Abstract class	Interface
Inheritance	Single	Multiple
State	Can have non-static, non-final variables	All variables are implicitly 'public final'.
Constructors	Can have constructor	Can't have constructor.

When we want to share code and instance variable among closely related classes, we prefer to use Abstract class. It implies a strict parent-child relationship.

When we need to support multiple inheritance or define a specific behaviour that unrelated class can implement.

Q3 | How does encapsulation ensure data security and integrity? Show with a Bank Account class using private variables and validate methods such as setAccountNumber that rejects null.

Am: Encapsulation ensures data security and integrity by restricting direct access to an object's internal components and forcing all interaction through controlled public methods.

I Ensuring data security: By declaring class variables as private, we prevent unauthorized external classes from viewing or modifying data directly. The internal representation is hidden.

II Ensuring data integrity: By providing public setter methods, we can apply logic to validate data before assigning it. This ensures that the object never enters an invalid state.

Code

```
Class BankAccount {
```

```
    private String AccountNumber;
```

```
    private double balance;
```

```
    public void setAccountNumber(String accNum) {
```

```
        if (accNum == null || accNum.isEmpty()) {
```

```
            System.out.println("Error, Account Number Can't be Empty");
```

```
        } else {
```

```
            this.AccountNumber = accNum;
```

```
            System.out.println("Success: Account Number set");
```

```
        }
```

```
    public void setInitialBalance(double amount) {
```

```
        if (amount < 0) {
```

```
            System.out.println("Error: Balance can't be neg");
```

```
        } else { this.balance = amount;
```

```
            System.out.println("Balance set to " + this.balance);
```

```
        }
```

```
    public double getbalance() { return balance; }
```

```
}
```

5

```

public class Encapsul {
    public static void main(String[] args) {
        BankAccount myAccount = new BankAccount();
        myAccount.setAccountNumber(""); //Failed Error
        myAccount.setInitialBalance(-500); //Error
        myAccount.setAccountNumber("SK1234");
        myAccount.setInitialBalance(1000);
    }
}

```

Output:

Error, Account Number can't be Empty

Error, Balance can't be neg

Success, Account Number set

Balance set to 1000.

Q5] Developing a multithreading-based project to simulate a car parking management system with classes namely RegistrarParking . . .

→ CODE

```
import java.util.LinkedList;  
import java.util.Queue;
```

```
Class RegistrarParking {  
    private String licensePlate;  
    public RegistrarParking (String licensePlate) {  
        this.licensePlate = licensePlate;  
    }  
    public String getLicensePlate() { return licensePlate; }  
}
```

```
Class ParkingPool {  
    private Queue <RegistrarParking> queue = new LinkedList<>();  
    public synchronized void addRequest(RegistrarParking car) {  
        queue.add(car)  
        System.out.println("Car " + car.getLicensePlate() + " requested  
        parking.");  
        notify();  
    }  
}
```

```

public synchronized RegistrarParking getRequest() {
    while (queue.isEmpty()) {
        try { wait(); }
        catch (InterruptedException e) { e.printStackTrace(); }
    }
    return queue.poll();
}
}

```

```

class ParkingAgent extends Thread {
    private ParkingPool pool;
    private int agentId;

    public ParkingAgent(ParkingPool pool, int agentId) {
        this.pool = pool;
        this.agentId = agentId;
    }
}

```

@Override

```

public void run() {
    while (true) {
        RegistrarParking car = pool.getRequest();
        System.out.println("Agent " + agentId + " parked car "
            + car.getLicensePlate() + ".");
        try { Thread.sleep(1000); }
        catch (InterruptedException e) {}
    }
}
}

```

```

public class ParkingSystem {
    public static void main (String [] args) {
        parkingPool pool = new ParkingPool ();
        new ParkingAgent (pool, 1).start ();
        new ParkingAgent (pool, 2).start ();

        String [] plates = { "ABC1", "ABC2", "ABC3" };
        for (String plate : plates) {
            new Thread () -> {
                pool.addRequest (new RegistrarParking (plate));
                .start ();
                try { Thread.sleep (200); } catch (Exception e) {}
            }
        }
    }
}

```

Output:

Car ABC1 requested parking.

Car ABC2 requested parking.

Agent1 parked car ABC1.

Agent2 parked car ABC2.

Q12] Describe how JDBC manages communication betⁿ a Java application and a relational database. Outline the steps involved in executing a 'SELECT' query and Fetching results. Include try-catch.

Ans: JDBC Communication Architecture JDBC acts as a bridge betⁿ Java application and a database.

- Java application: Calls JDBC API methods.
- JDBC Manager: Manages a list of database driver
- JDBC driver: Translates the standard JDBC calls into the specific protocol required by the database.

CODE:

```
import java.sql.*;
```

```
public class JDBC {
```

```
    public static void main(String[] args) {
```

```
        String url = "jdbc:mysql://localhost:3306/testdb";
```

```
        String user = "root"
```

```
        String password = "1234"
```

```
        Connection conn = null;
```

```
        Statement stmt = null;
```

```
        ResultSet rs = null;
```

try {

```
conn = DriverManager.getConnection(url, user, password);
```

```
System.out.println("Connected to Database");
```

```
stmt = conn.createStatement();
```

```
String sql = "Select * from students";
```

```
rs = stmt.executeQuery(sql);
```

```
while(rs.next()) {
```

```
    int id = rs.getInt("id");
```

```
    String name = rs.getString("name"); }
```

```
} catch (ClassNotFoundException) {
```

```
    System.out.println("JDBC not found");
```

```
} finally {
```

```
    try { if(rs != null) rs.close();
```

```
        if(stmt != null) stmt.close();
```

```
        if(conn != null) conn.close();
```

```
        System.out.print("Database closed");
```

```
} catch (SQLException e) { e.printStackTrace();
```

```
}
```

```
}
```

```
}
```

Output:

Connected to Database

101 Sajeeb

Database closed.

Q17 In a Java EE application, how does a servlet controller manage the flow between the model and the view? Provide a brief example that demonstrates forwarding data from a servlet to a JSP and rendering a response?

Ans: 1) The controller's Role in Java EE, the Servlet acts as the controller in the MVE architecture. It manages the flow using the following steps:

a) Interception: The servlet intercepts the incoming HTTP request.

b) Model Interaction: It calls business logic or creates Java Objects to process data.

c) Data Attachment: It attaches this processed data to the request object using `request.setAttribute()`.

d) View Selection: It selects the appropriate JSP page and forwards the request using the `RequestDispatcher`.

2) Key Mechanism: Forwarding: The servlet doesn't generate the HTML. Instead, it hands over the request and response objects to the JSP using `forward` method.

13

```
request.setAttribute("currentUser", student);
```

```
RequestDispatcher dispatcher = request.getRequestDispatcher
```

```
Dispatcher("profile.jsp");
```

```
dispatcher.forward(request, response);
```

```
§
```

profile.jsp

```
<%@ page contentType="text/html; charset=UTF-8"%>
```

```
<html>
```

```
<head>
```

```
<title> User Profile </title>
```

```
</head>
```

```
<body>
```

```
<h2> User Details </h2>
```

```
<p> Name: $ { currentUser.name } </p>
```

```
<p> ID: $ { currentUser.id } </p>
```

```
</body>
```

```
</html>
```

Output

User Details

Name: Sajeeb

ID: 101

Q22 How does Prepared Statement improve performance and security over statement in JDBC? Write a short example to insert a record into a MySQL table using prepared statement.

Ans:

~~\$~~ Performance Improvement:

1) Statement: Every time we run a statement, the database must parse, compile, and query plan from scratch.

2) Prepared Statement: The query is pre-compiled by the database once. When we run it multiple time, the database reuses the existing plan, just swapping the variables. This is significantly faster for repeated executions.

Security Improvement:

1) Statement: It concatenates string directly. So its easy to hack the database.

2) Prepared Statement: It uses placeholders (?). The database treats user input strictly as data, not executable code. This neutralizes malicious SQL commands.

CODE

```
import java.sql.*;

public class SQL {

    public static void main(String[] args) {

        String url = "jdbc:mysql://localhost:3306/testdb";
        String user = "root";
        String pass = "1234";

        String sql = "Insert into students (name, email) values (?, ?)";

        try {
            Connection conn = DriverManager.getConnection(url, user, pass);
            PreparedStatement stmt = conn.prepareStatement(sql);

            stmt.setString(1, "Sajeeb");
            stmt.setString(2, "sajeeb@mbatu.ac.bd");

            stmt.executeUpdate();
            System.out.println("Success");
            conn.close();
        } catch (Exception e) { e.printStackTrace(); }
    }
}
```

✂

Output:

Success

Q1 What is a ResultSet in JDBC and how is it used to retrieve data from MySQL? Briefly explain the use of `next()`, `getString()`, `getInt()` methods with an example.

Ans:

A ResultSet is an interface in the `java.sql` package that represents the table of data generated by executing a database query. It acts as an iterator that maintains a cursor pointing to the current row of data. Initially the cursor is positioned before the first row.

⇒ `next()`: Moves the cursor forward one row from its current position. It returns true if the new current row is valid. It returns false if there is no more rows. It is typically used in a while loop to iterate through data.

⇒ `getInt()`: Retrieves the value of the specified column in the current row as Java int.

⇒ `getString()`: Retrieves the value of the specified column in the current row as a Java String.

CODE:

```
import java.sql.*;

public class SQL {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/testdb";
        String user = "root";
        String pass = "1234"

        try {
            Connection conn = DriverManager.getConnection(url, user,
                                                            pass);

            Statement stmt = conn.createStatement();
            String sql = "Select * from students";
            ResultSet rs = stmt.executeQuery(sql);

            while (rs.next()) { int id = rs.getInt('Id');
                               String name = rs.getString('name');

                               System.out.println("ID: " + id + " Name: " + name);
                               }
            conn.close();
        } catch (Exception e) { e.printStackTrace(); }
    }
}
```

Output:

ID: 101

SA Name: Sajeeb

Q 26 | Design a simple CRUD application using Spring Boot and MySQL to manage student records. Describe how each operation would be implemented using a repository interface.

Ans:

To design a CRUD application in Spring Boot, we use Spring Data JPA. This abstraction layer eliminates the need for boilerplate JDBC code. The core component is the Repository Interface, which acts as a bridge between the Java application and the MySQL database.

We define an interface that extends JpaRepository. This automatically provides methods for all standard database operations without writing any SQL.

Code:

```
import org.springframework.data.jpa.repository.JpaRepository;

public interface StudentRepository extends JpaRepository
    < Student, Long > {
    // CRUD methods here
}
```

10

Here is how each operation is handled using the repository methods:

i) Create (Insert): We use the `.save(entity)` method.

If the entity's primary key is null, hibernate treats it as a new record and executes an SQL INSERT.

ii) Read (Select): `repo.save(new student("Sajeesh", "ICT"))`

We use `.findAll()` to retrieve a list of all records or `.findById(id)` to find a specific record.

```
List<student> list = repo.findAll();
```

iii) Update: We also use the `.save(entity)` method.

However if the entity object contains an ID that already exists in database, hibernate performs an SQL 'update' instead of 'Insert'.

```
Student s = repo.findById(11).get();  
s.setName("New Name");  
repo.save(s);
```

iv) Delete: we use `.deleteById(id)` to remove a record based on its primary key.

```
repo.deleteById(104);
```

// Full CODE

@RestController

@RequestMapping("/students")

public class StudentController {

@Autowired

private StudentRepository repo;

@PostMapping // Create

```

    public Student createStudent(@RequestBody Student s) {
        return repo.save(s);
    }

```

@GetMapping // Read

```

    public List<Student> getAllStudents() {
        return repo.findAll();
    }

```

@PutMapping("/{id}") // UPDATE

```

    public Student updateStudent(@PathVariable Long id,
        @RequestBody Student s) {
        s.setId(id);
        return repo.save(s);
    }

```

@DeleteMapping("/{id}")

```

    public String deleteStudent(@PathVariable Long id) {
        repo.deleteById(id);
        return "Deleted id: " + id;
    }
}

```

21

Q 27] How does SpringBoot simplify the development of RESTful services? Describe how to implement a REST Controller using @RestController, @GetMapping and @PostMapping, including JSON data handling.

Ans:

Spring boot drastically reduces the complexity of setting up REST APIs compared to legacy Spring MVC.

I] Auto-Configuration: It automatically configures the underlying Spring beans, dispatcher and view resolvers, eliminating the need for complex XML files.

II] Embedded Server: It comes with an embedded Tomcat Server, so developers don't need to install or configure an external web server to run the application.

III] Starter Dependencies: The spring-boot-starter-web dependency includes all necessary libraries in a single package.

- ⇒ @RestController: This is a convenience annotation that combines @Controller and @ResponseBody. It indicates that the class is a web controller and that the return value of its methods should be written directly to the HTTP response body rather than resolving to a JSP/HTML view.
- ⇒ @GetMapping: Used to map HTTP Get requests to a specific handler method. It is generally used for fetching/reading data.
- ⇒ @PostMapping: Used to map HTTP post requests to a specific handler method. It is generally used for creating/submitting new data.

JSON data handling:

Spring boot uses the Jackson library to handle JSON conversion automatically.

- Input (Request): The @RequestBody annotation deserializes incoming JSON data into a Java object.
- Output (Response): When a Java object is returned from a method, Spring automatically serializes it into

JSON format.

CODE:

```
import org.springframework.web.bind.annotation.*;
import java.util.*;
```

@RestController

@RequestMapping("/api/students")

public class StudentController {

```
    private List<String> students = new ArrayList<>() {
        Arrays.asList("Rahim", "Karim");
    };
```

@GetMapping

```
public List<String> getAllStudents() {
    return students;
}
```

@PostMapping

```
public String addStudent(@RequestBody String name) {
```

```
    Student.add(name);
```

```
    return "Success: " + name + " added.";
}
```

```
} }
```